

TAKE-HOME ASSIGNMENT

Wallet Ledger Backend Service

AI Engineer · Java / Spring Boot · GitHub Submission

TIME EXPECTATION

4-8 hours

(submit within 3-5 days)

SUBMISSION

GitHub Link

Public or private repo

STACK

Spring Boot

Java + PostgreSQL / H2

1. Project Goal

Build a production-minded **Wallet Ledger backend service** in Java + Spring Boot.

This assignment is **intentionally open-ended**. You decide the exact architecture, package structure, API design, and any extra features — as long as the mandatory capabilities listed below are correctly implemented and can be demonstrated through working code.

We want to see how you design and implement a real backend service that safely handles money movement and keeps a clear record of every change.

2. Technology Baseline

Required	Java 17+, Spring Boot 3.x, Relational DB (PostgreSQL preferred, H2 acceptable)
Optional	Redis, Docker / Docker Compose, OpenAPI, WebSocket, event publishing, Flyway/Liquibase, etc.

3. Mandatory Capabilities

Your service must correctly support the following. How you design and implement them is up to you.

3.1 Wallet Operations

The service must be able to:

- Credit a player's wallet
- Debit a player's wallet
- Reject a debit when the player does not have enough balance
- Return the current balance of a player
- Return a player's transaction history (paginated)

Every balance change must leave a clear, permanent record of what happened and why (e.g. linked to a mission reward, purchase, or admin action).

3.2 Safety & Correctness

In a production wallet these behaviours matter:

- The same request must not be applied more than once
- Two requests arriving at the same time must not leave the wallet in an incorrect state
- A failed operation must not leave partial updates
- Invalid inputs (e.g. negative amounts, missing player) should be handled clearly

3.3 Tests

Include automated tests that give confidence the service behaves correctly under normal use and under pressure (for example, concurrent requests or repeated submissions of the same request).

You decide the test structure and coverage. We will look at whether the important money paths are actually protected.

3.4 README

Explain:

- How to run the project and the tests
- Your design decisions and the main trade-offs you made
- Any assumptions or known limitations

4. Additional Aspects We Value

The following are optional. You may implement any that you find relevant to a production wallet service.

4.1 Suggested Product Features

You may extend the service with any of the following product features if you wish:

Feature	Product need
Daily Login Streak	Players receive increasing rewards for consecutive daily logins. Missing a day resets the streak.
Limited Promotional Reward	A special reward that can only be claimed by a limited number of players (for example, the first 100).
Player Currency Transfer	One player can send in-game currency to another player.
Transaction Refund	Support reversing a previous wallet transaction when a purchase or reward is cancelled.
Reservation of Funds	Temporarily hold an amount on a player's balance (e.g. when placing a bid), then later finalise or release it.
Bulk Reward Distribution	Credit rewards to a large group of players at once (for example after a tournament).
Claim Reward	Client reports that an action is completed; the server decides whether the reward should be granted.

4.2 Supporting Practices

You may also include any of the following if useful:

Practice	Description
Domain Events	Notify other parts of the system when a wallet balance changes
Docker Compose	Run the application and database with a single command
OpenAPI / Swagger	Interactive API documentation for the service
Balance Check	A way to verify that current balances still match the full transaction history
AI Tooling Notes	Brief note on any AI coding tools used while building the project

Note: These aspects are only considered after the mandatory capabilities are solid. A clean and correct core implementation is always more valuable than incomplete extras.

5. Open-Ended Aspects

Beyond the mandatory and bonus items, you are free to add anything that demonstrates strong backend service design, for example:

- Clean REST controllers with proper validation and error responses
- Redis caching for getBalance
- Basic observability (structured logging, metrics)
- Any other feature that shows how you handle real-world dynamic tasks

6. README Requirements

Your README must include the following sections:

- **How to run** — project setup, database, and how to execute the tests
- **Design decisions** — chosen ledger approach and the main trade-offs
- **Concurrency & Idempotency** — how you handled both
- **Testing approach** — especially the concurrent debit case
- **Assumptions & limitations** — known gaps or future improvements

7. Submission

1. Push the complete project to a **GitHub repository**.
2. The repository may be **public or private**. If private, grant access to the reviewer.

3. Submit only the **GitHub repository link**.
4. Ensure the README is complete enough that a reviewer can understand your design and run the project without additional explanation.

8. Evaluation Criteria

Area	What we look for
Money-moving correctness	Correct wallet behaviour under normal and concurrent use
Service design	Clean structure, readable code, sensible layering
Concurrency & edge cases	Safe behaviour when requests overlap or fail
Test quality	Tests that protect the important money paths
Documentation	Clear README and design reasoning
Optional aspects	Only considered after mandatory capabilities are solid

— END OF ASSIGNMENT —

Submit your GitHub repository link when ready. Good luck.